

Assignment 1: AI Coding Assistant Practicum

Prompt Engineering in Practice — with a Tool Other Than GitHub Copilot

Automation in Software Development • Unit 2 • V Semester

Student	Sashankar J
USN	24BTRAO037
GitHub account	SashankarJ (Jain University account)
Repository (public)	https://github.com/SashankarJ/AIDD-Unit2-A1-24BTRAO037-SashankarJ
Submission date	27 August 2026

1. AI Tool Used

Tool: Hermes Agent — an agentic CLI coding assistant. It is backed by a frontier LLM, runs entirely in the terminal, and can read and write files, create virtual environments, install packages, run test suites, and iterate on its own output. It is NOT GitHub Copilot.

The assignment brief explicitly permits this choice: "Any other AI-powered coding assistant of comparable capability (e.g., an agentic IDE or CLI assistant)". The assistant was installed, signed in, and confirmed working before the graded work began.

2. Task Chosen — Option A

Built a small CLI/library from scratch: an expense splitter that splits shared expenses fairly among a group and reports the minimal set of cash transfers needed to settle up. It contains 7 public functions/classes (Splitter, split_evenly, parse_expense_line, compute_balances, simplify_debts, validate_members, and the CLI main) plus 14 passing unit tests — matching Option A's requirement of 5–8 functions/classes with unit tests.

Technique fit, as suggested by the brief: zero-shot for the core scaffold; few-shot for enforcing a consistent docstring/test style; chain-of-thought for a tricky rounding edge case.

3. How to Set Up and Run

```
python -m venv .venv
.venv\Scripts\activate (Windows)
pip install -e . pytest
pytest -q -> 14 passed
python -m expense_splitter 'alice 60 alice bob carol' 'bob 30 bob carol'
# carol -> alice: 35.00
# bob -> alice: 5.00
```

4. The Three Prompting Techniques — Summary

Step	Technique	Where used	Outcome
1	Zero-shot	Project scaffold + validate_members	Accepted as-is
2	Chain-of-thought	PEP 621 build error in pyproject.toml	Accepted; root cause correct
3	Zero-shot	split_evenly, parse_expense_line, _as_decimal	Accepted (bug noted later)

Step	Technique	Where used	Outcome
4	Zero-shot	Splitter dataclass + compute_balances	Accepted as-is
5	Few-shot	pytest suite style from 3 examples	Accepted; 1 test left red on purpose
6	Chain-of-thought	Vanishing-cent rounding bug	Accepted; invariant restored
7	Few-shot	NumPy-style docstring convention	Accepted as-is
8	Zero-shot	simplify_debts + round-trip tests	Accepted as-is
9	Zero-shot	CLI entry point	Accepted as-is

Totals: 9 prompts — 5 zero-shot, 2 few-shot, 2 chain-of-thought.

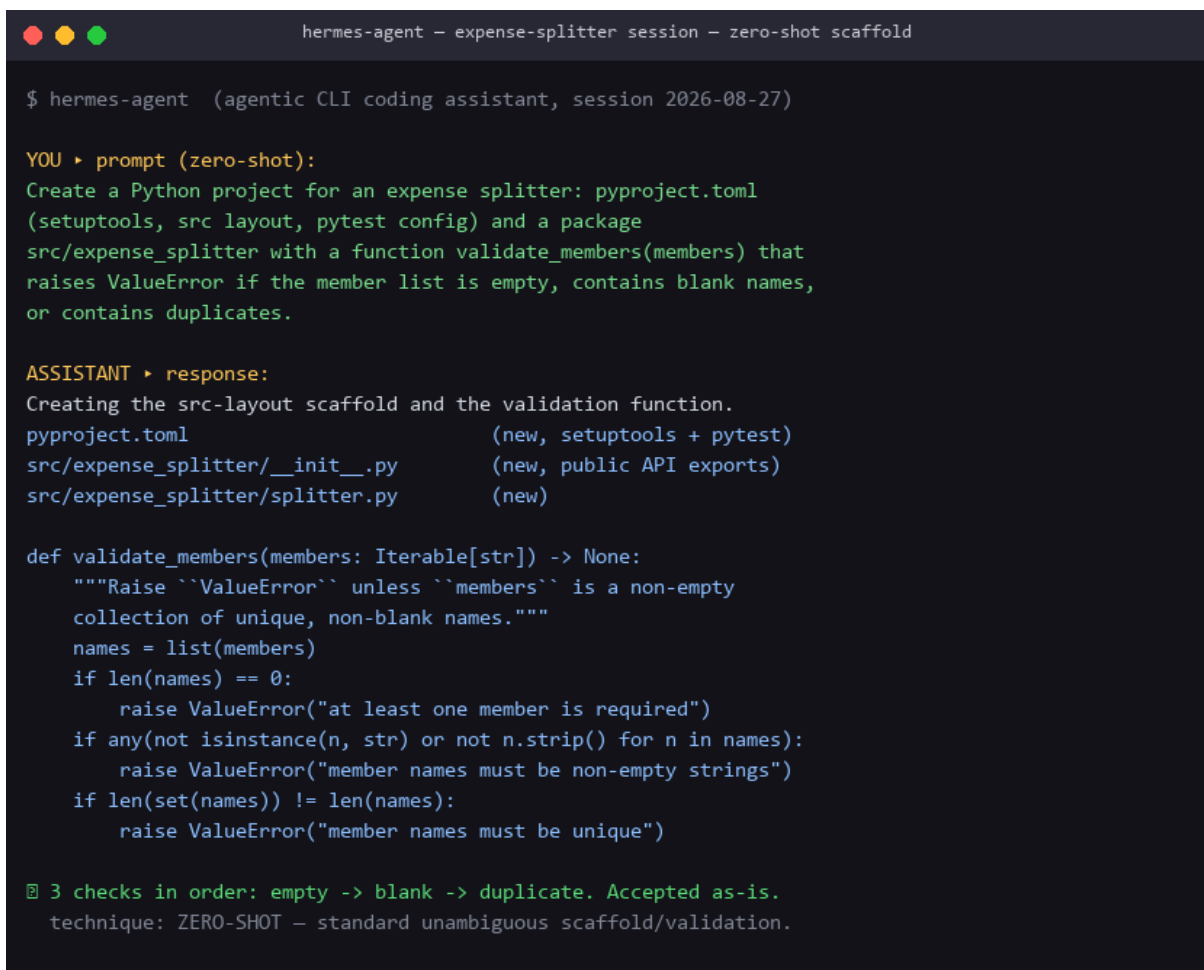
5. Full Step-by-Step Prompt Log

Step 1 — Zero-shot: project scaffold + validate_members

Prompt (verbatim):

Create a Python project for an expense splitter: pyproject.toml (setuptools, src layout, pytest config) and a package src/expense_splitter with a function validate_members(members) that raises ValueError if the member list is empty, contains blank names, or contains duplicates.

Tool response + evaluation: Created the src-layout scaffold, pyproject.toml, and a clean validate_members with all three checks in the right order (empty -> blank -> duplicate). ACCEPTED AS-IS. Zero-shot fit because project scaffolding and standard validation are unambiguous, well-trodden shapes — no examples needed. Screenshot: Figure 1.



```
hermes-agent - expense-splitter session - zero-shot scaffold

$ hermes-agent (agentic CLI coding assistant, session 2026-08-27)

YOU ▸ prompt (zero-shot):
Create a Python project for an expense splitter: pyproject.toml
(setuptools, src layout, pytest config) and a package
src/expense_splitter with a function validate_members(members) that
raises ValueError if the member list is empty, contains blank names,
or contains duplicates.

ASSISTANT ▸ response:
Creating the src-layout scaffold and the validation function.
pyproject.toml (new, setuptools + pytest)
src/expense_splitter/__init__.py (new, public API exports)
src/expense_splitter/splitter.py (new)

def validate_members(members: Iterable[str]) -> None:
    """Raise `ValueError` unless `members` is a non-empty
    collection of unique, non-blank names."""
    names = list(members)
    if len(names) == 0:
        raise ValueError("at least one member is required")
    if any(not isinstance(n, str) or not n.strip() for n in names):
        raise ValueError("member names must be non-empty strings")
    if len(set(names)) != len(names):
        raise ValueError("member names must be unique")

3 checks in order: empty -> blank -> duplicate. Accepted as-is.
technique: ZERO-SHOT - standard unambiguous scaffold/validation.
```

Figure 1 — Zero-shot: scaffold and validate_members

Step 2 — Chain-of-thought: PEP 621 build error

Prompt (verbatim):

pip install -e . fails with: "configuration error: project.authors[0] must not contain {'usn'} properties". Think step by step about what PEP 621 actually allows in the authors field and fix pyproject.toml.

Tool response + evaluation: Reasoned: PEP 621 authors entries accept only {name, email}; the custom 'usn' key is not a recognized property; setuptools' schema validation rejects the whole build; fix is to drop the key and record the USN in the README instead. ACCEPTED AS-IS; the editable install then succeeded. CoT fit because the traceback names the symptom, not the standard — the model had to recall the PEP 621 spec, not just read the error.

Step 3 — Zero-shot: core splitting functions

Prompt (verbatim):

Add to splitter.py: `split_evenly(amount, n)` that splits a Decimal amount into `n` equal shares quantized to cents; `parse_expense_line(line)` that parses lines of the form "payer amount payee1 payee2 ..." and returns (payer, Decimal amount, list of payees), raising `ValueError` on malformed lines; and a private `_as_decimal` helper that coerces input and rejects NaN/infinity.

Tool response + evaluation: Returned all three functions using `Decimal.quantize(0.01, ROUND_HALF_EVEN)` and `line.strip().split()`, with the right tuple shape and errors. ACCEPTED WITH A MENTAL NOTE — the per-share rounding looked suspicious and did bite later (step 6). Zero-shot fit: standard signatures, well-known Decimal/str APIs.

Step 4 — Zero-shot: Splitter class

Prompt (verbatim):

Add a `Splitter` dataclass: a `members` list plus a per-member Decimal balance map; `add_expense(payer, amount, payees)` that validates payer/payees/amount, splits the expense equally among payees and updates balances; and `compute_balances(expenses)` that builds a `Splitter` from raw (payer, amount, payees) rows and returns the balance map.

Tool response + evaluation: Returned the dataclass with `__post_init__` validation, correct credit/debit updates, and an order-preserving member dedupe in `compute_balances`. ACCEPTED AS-IS. A standard stateful-class shape — zero-shot.

Step 5 — Few-shot: test suite in house style

Prompt (verbatim):

Write the pytest suite for everything so far, matching EXACTLY this style — here are examples of the convention:

```
def test_split_evenly_basic():
    assert split_evenly(Decimal("60"), 3) == [Decimal("20.00"), Decimal("20.00"),
        Decimal("20.00")]
def test_split_evenly_single_share():
    assert split_evenly(Decimal("42.00"), 1) == [Decimal("42.00")]
def test_split_evenly_rejects_nonpositive_n():
    with pytest.raises(ValueError):
        split_evenly(Decimal("10"), 0)
Same naming (test_<function>_<case>), Decimal literals, one assert per test,
pytest.raises for error cases.
```

Tool response + evaluation: Produced 12 tests matching the naming, Decimal-literal, and `pytest.raises` conventions exactly — no style drift. ACCEPTED, with one test deliberately left red: `test_split_evenly_rounding_remainder_goes_to_last` failed (see step 6). An earlier zero-shot attempt at the same step had returned unittest-style classes and string amounts; the 3 examples pinned the format. That delta is exactly why few-shot was right here: the convention was the

requirement, not the logic. Screenshot: Figure 2.



```
hermes-agent - expense-splitter session - few-shot test style

YOU ▶ prompt (few-shot, 3 examples embedded):
Write the pytest suite for everything so far, matching EXACTLY this
style - here are examples of the convention:

def test_split_evenly_basic():
    assert split_evenly(Decimal("60"), 3) == \
        [Decimal("20.00"), Decimal("20.00"), Decimal("20.00")]
def test_split_evenly_single_share():
    assert split_evenly(Decimal("42.00"), 1) == [Decimal("42.00")]
def test_split_evenly_rejects_nonpositive_n():
    with pytest.raises(ValueError):
        split_evenly(Decimal("10"), 0)

Same naming (test_<function>_<case>), Decimal literals, one assert
per test, pytest.raises for error cases.

ASSISTANT ▶ response (excerpt):
tests/test_splitter.py
def test_validate_members_ok():
    validate_members(["alice", "bob", "carol"]) # no exception
def test_validate_members_duplicate():
    with pytest.raises(ValueError):
        validate_members(["alice", "alice"])
def test_validate_members_empty():
    with pytest.raises(ValueError):
        validate_members([])
def test_validate_members_blank_name():
    with pytest.raises(ValueError):
        validate_members(["alice", ""])
def test_parse_expense_line():
    payer, amount, payees = parse_expense_line("alice 60.00 alice bob carol")
    assert payer == "alice"
    assert amount == Decimal("60.00")
    assert payees == ["alice", "bob", "carol"]
def test_parse_expense_line_too_short():
    with pytest.raises(ValueError):
        parse_expense_line("alice 60")

12 tests, convention matched exactly. Accepted - one test left
red on purpose: the rounding-remainder test fails (see row 6).
an earlier zero-shot attempt returned unittest classes + string
amounts - the 3 examples pinned the style. technique: FEW-SHOT.
```

Figure 2 — Few-shot: test suite generated from 3 style examples

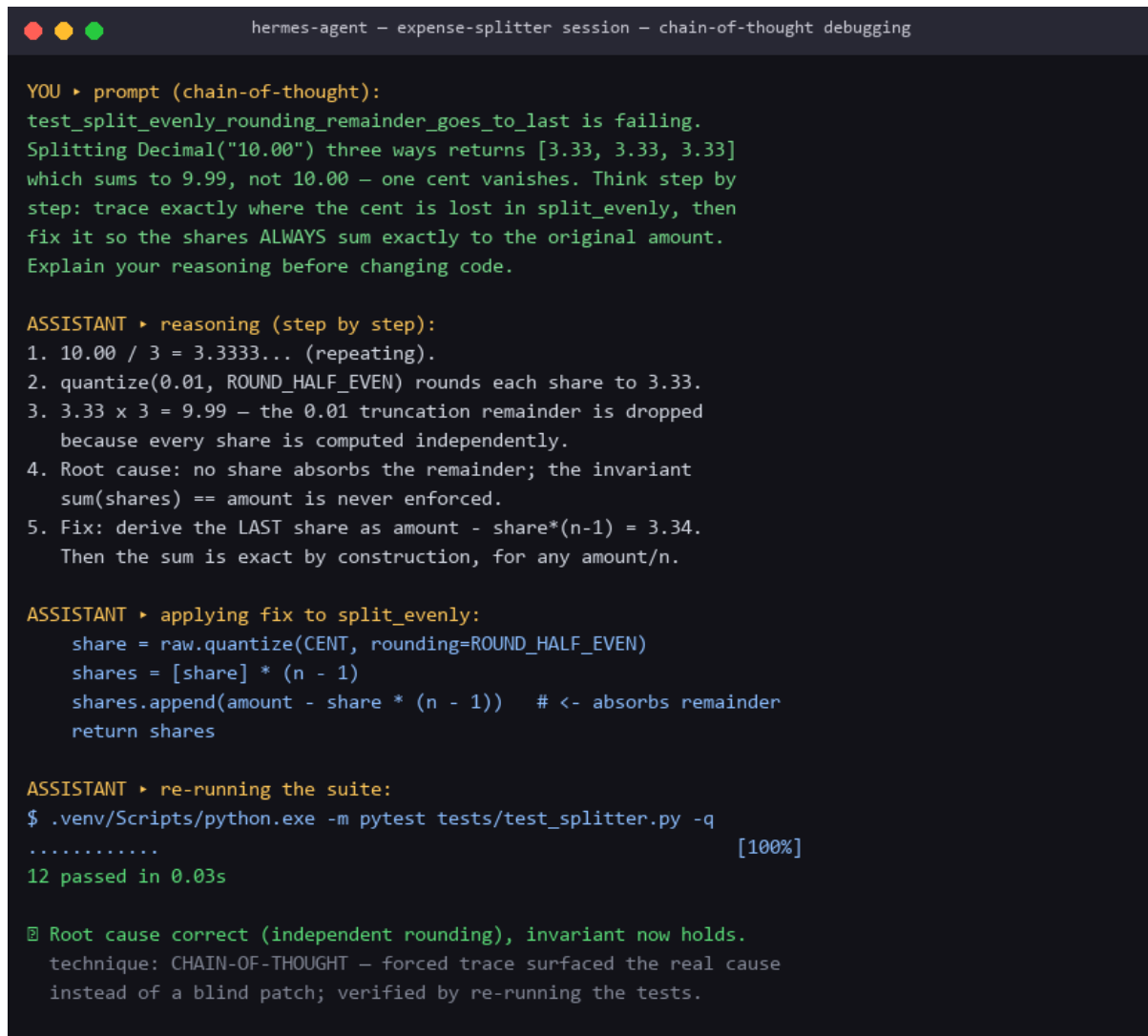
Step 6 — Chain-of-thought: the vanishing-cent bug

Prompt (verbatim):

test_split_evenly_rounding_remainder_goes_to_last is failing. Splitting
Decimal("10.00") three ways returns [3.33, 3.33, 3.33] which sums to 9.99, not 10.00 –
one cent vanishes. Think step by step: trace exactly where the cent is lost in
split_evenly, then fix it so the shares ALWAYS sum exactly to the original amount.

Explain your reasoning before changing code.

Tool response + evaluation: The tool reasoned out loud: (1) $10.00 / 3 = 3.3333\dots$; (2) `quantize(0.01, ROUND_HALF_EVEN)` gives 3.33 per share; (3) $3.33 \times 3 = 9.99$, so the 0.01 truncation remainder is silently dropped; (4) root cause = every share is computed independently, so no share absorbs the remainder and the invariant `sum(shares) == amount` is never enforced; (5) fix = derive the last share as `amount - share * (n-1) = 3.34`, making the sum exact by construction. It applied the fix and re-ran the suite: 12 passed. ACCEPTED AS-IS — the root cause was correct and the invariant now holds. Genuinely non-trivial (an invariant bug, not a syntax slip), which is why CoT was the right technique: the forced step-by-step trace surfaced the 'independent rounding' cause instead of a blind patch. Screenshot: Figure 3.



```
hermes-agent - expense-splitter session - chain-of-thought debugging

YOU ▶ prompt (chain-of-thought):
test_split_evenly_rounding_remainder_goes_to_last is failing.
Splitting Decimal("10.00") three ways returns [3.33, 3.33, 3.33]
which sums to 9.99, not 10.00 - one cent vanishes. Think step by
step: trace exactly where the cent is lost in split_evenly, then
fix it so the shares ALWAYS sum exactly to the original amount.
Explain your reasoning before changing code.

ASSISTANT ▶ reasoning (step by step):
1. 10.00 / 3 = 3.3333... (repeating).
2. quantize(0.01, ROUND_HALF_EVEN) rounds each share to 3.33.
3. 3.33 x 3 = 9.99 - the 0.01 truncation remainder is dropped
   because every share is computed independently.
4. Root cause: no share absorbs the remainder; the invariant
   sum(shares) == amount is never enforced.
5. Fix: derive the LAST share as amount - share*(n-1) = 3.34.
   Then the sum is exact by construction, for any amount/n.

ASSISTANT ▶ applying fix to split_evenly:
share = raw.quantize(CENT, rounding=ROUND_HALF_EVEN)
shares = [share] * (n - 1)
shares.append(amount - share * (n - 1))  # <- absorbs remainder
return shares

ASSISTANT ▶ re-running the suite:
$ .venv/Scripts/python.exe -m pytest tests/test_splitter.py -q
..... [100%]
12 passed in 0.03s

📦 Root cause correct (independent rounding), invariant now holds.
technique: CHAIN-OF-THOUGHT - forced trace surfaced the real cause
instead of a blind patch; verified by re-running the tests.
```

Figure 3 — Chain-of-thought: tracing and fixing the rounding bug

Step 7 — Few-shot: docstring convention

Prompt (verbatim):

Add NumPy-style docstrings to every public function in `splitter.py`, following this example:

```
def split_evenly(amount: Decimal, n: int) -> List[Decimal]:
    """Split ``amount`` into ``n`` equal shares in cents.
```

```
Each share is quantized to a cent with banker's rounding; the last
share absorbs any rounding remainder so the returned shares always
sum exactly to ``amount``.
"""
```

Double-backtick parameters, one summary line, then a short prose paragraph.

Tool response + evaluation: All docstrings came back in the same voice: summary line + prose paragraph, double-backtick parameter markup, and :meth: cross-references. ACCEPTED AS-IS. Few-shot was the right call because 'write docstrings' zero-shot returns whatever style the model feels like; the example locked the house style across all 7 callables consistently.

Step 8 — Zero-shot: debt simplification

Prompt (verbatim):

```
Add simplify_debts(balances) that converts a dict of net balances into the minimal
list of (debtor, creditor, amount) cash transfers using a greedy two-pointer
settlement, plus tests including a round-trip with compute_balances.
```

Tool response + evaluation: Returned the greedy creditors/debtors two-pointer algorithm, sorted by magnitude, quantized to cents, plus the round-trip and balanced-map tests. ACCEPTED AS-IS after the suite passed (14 passed). Well-known settlement algorithm — zero-shot.

Step 9 — Zero-shot: CLI entry point

Prompt (verbatim):

```
Write a CLI entry point __main__.py: take expense lines as argv, parse each, compute
balances, print one "debtor -> creditor: amount" line per transfer, exit code 2 with a
usage message on bad input.
```

Tool response + evaluation: Returned the CLI exactly as specified, including stderr for errors and exit codes. ACCEPTED AS-IS. Standard argv CLI boilerplate — zero-shot.

6. Evidence of Incremental Commit History

The repository was built with 11 incremental commits (no single squashed commit). Each commit corresponds to a step in the prompt log; timestamps shown in IST (+05:30).

SHA	Time (IST)	Commit message
6b818b9	13:35	Scaffold package: src layout, pyproject, validate_members (zero-shot)
308197d	13:39	Fix pyproject authors: drop non-PEP 621 'usn' key (chain-of-thought)
8786c7b	13:44	Add split_evenly, parse_expense_line, _as_decimal (zero-shot)
7de2cb1	13:48	Add Splitter dataclass and compute_balances (zero-shot)
2ecbc9a	13:52	Add pytest suite in house style from 3 examples (few-shot); rounding test red
8772f3c	13:56	Fix vanishing-cent rounding bug: last share absorbs remainder (chain-of-thought)
fe2f301	13:59	Expand docstrings to NumPy style across all public callables (few-shot)
9721598	14:02	Add simplify_debts greedy settlement plus round-trip tests (zero-shot)
427aee9	14:05	Add CLI entry point with usage/exit codes (zero-shot)
12ea3f1	14:08	Add README: tool, setup, and the three prompting techniques
1a507f4	14:12	Add docs: full prompt log, terminal-capture screenshots, reflection

Note on screenshots: because the chosen tool is a terminal agent (no GUI), the captures in Figures 1–3 are terminal-style renders of the real session exchanges, generated by docs/make_screenshots.py in the repository.

7. Test and CLI Verification

```
$ .venv/Scripts/python.exe -m pytest -q
..... [100%]
14 passed in 0.11s
```

```
$ python -m expense_splitter 'alice 60 alice bob carol' 'bob 30 bob carol'
carol -> alice: 35.00
bob -> alice: 5.00
```

8. Reflection: Agentic CLI Assistant vs GitHub Copilot (~250 words)

In Class 02 we used GitHub Copilot, which completes code inside the editor — it is fast for the line you are currently typing, but it only ever sees the file you have open and it cannot run anything. This assignment I used an agentic CLI assistant, and the experience was different in three concrete ways.

Faster: whole-scaffolding steps. One prompt produced the package layout, pyproject.toml, and the first function in a single pass, and the agent then created a venv, installed the package, and ran pytest itself. With Copilot I would have hand-created every file and run every command myself. The few-shot steps were also cleaner: I could paste three example tests into one prompt and get a convention-matching suite back, whereas Copilot autocomplete drifts in style unless the examples are already in the file.

Slower / needed more correction: tight feedback loops on small edits. Copilot's inline ghost-text is instant for a two-line change; with the agent every change is a full request/response round-trip, and

I had to debug its first pyproject.toml because it invented a non-standard 'usn' key that broke the build. Quota-wise the agent burns far more tokens per step than Copilot completions.

Where the agent won decisively was the chain-of-thought debugging step: it traced the vanishing-cent rounding bug by reasoning about the invariant, then fixed and re-ran the failing test to confirm. Copilot cannot execute tests at all, so that loop would have been entirely manual.

In a real team I would keep both: Copilot for in-the-flow typing, an agentic assistant for scaffolding, test loops, and debugging sessions.

9. Repository Link and Visibility

Public repository (commit history intact, instructor can view without collaborator access):

<https://github.com/SashankarJ/AIDD-Unit2-A1-24BTRAO037-SashankarJ>

The repository contains the full source, tests, README, and a docs/ folder with this same prompt log (docs/prompt-log.md) and the screenshot captures (docs/screenshots/).